SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

This document introduces ChainLogic AI — an agentic supply chain decision intelligence system for automotive SMEs, as outlined in the Product Requirement Document. It provides a comprehensive reference for developers and testers on the system's architecture, data flows, model integration, and design decisions.

1.1. Purpose:

The SAD covers the following key elements:

1. *Architecture: A decoupled client-server system — a Next.js frontend communicating with a Python FastAPI backend, backed by a normalised SQLite ERP database and powered by Z.AI GLM as the reasoning service layer.*
2. *Data Flows: How a supplier disruption email travels from the browser UI through the agentic pipeline, gets fused with live ERP data, is reasoned over by Z.AI GLM, and returns a structured financial decision to the frontend — with the chosen action written back to SQLite.*
3. *Model Process: The end-to-end workflow of the two core features: Agentic Crisis Analysis and Closed-Loop ERP Execution.*
4. *Role of Reference: This document serves as the architectural ground truth for all team members during development, testing, and the demo presentation.*

1.2. Background:

Previous State: Automotive SME supply chain managers rely on fragmented manual workflows — reading supplier emails, manually cross-referencing spreadsheet inventory data, calling suppliers for quotes, and making intuition-based decisions under financial pressure. This process takes hours while daily downtime penalties compound (up to $85,000/day per production job).

What ChainLogic AI Changes: The system introduces a closed-loop AI decision layer on top of an existing ERP data model. It replaces manual analysis with a two-call Z.AI GLM agentic workflow, eliminates re-entry of decisions by writing back to the database autonomously, and provides full financial trade-off transparency through a structured UI.
New Capabilities Introduced:

- *Agentic SKU extraction from free-form email text (no manual field mapping)*
- *Real-time ERP data fusion into the GLM context window*
- *Automated financial trade-off generation with formula and math breakdown*
- *One-click closed-loop ERP execution with auditable DB updates*

© UMHackathon 2026

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
Supply Chain Manager	*Primary user — selects disruption emails, reviews trade-off options, approves execution*	*Fast, quantified decision support; no manual spreadsheet work; clear financial justification*
Operations Director	*Reviews decisions and financial exposure*	*High-level visibility of crisis status, risk in dollar terms, and which action was taken*
Procurement Officer	*Identifies alternative sourcing options*	*Multiple sourcing options with cost comparisons; no reliance on tribal knowledge*
SME Founder / CEO	*Approves strategic decisions*	*Single screen showing risk, options, and AI recommendation with clear dollar figures*
Development Team	*Builds and maintains the system*	*Clear API contracts, modular codebase, documented prompt architecture*
QA Team	*Validates system behaviour*	*Defined request/response schemas, edge case coverage, fallback behaviour specifications*

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Web Application (Browser-based, Desktop-first)*
Architecture	*Decoupled Client-Server with AI Service Layer*

ChainLogic AI is structured as a two-tier client-server system with an external AI service dependency. The Next.js React frontend (running on localhost:3000) serves as the single-page application for the supply chain manager. It communicates exclusively with the Python FastAPI backend (running on localhost:8000) via two REST endpoints. The backend owns all business logic: it orchestrates the Z.AI GLM API calls, reads from and writes to the SQLite ERP database, and returns structured JSON to the frontend.

Z.AI GLM (ilmu-glm-5.1 via https://api.ilmu.ai/v1/chat/completions) is consumed as an external AI service layer — it is not embedded in the backend but called synchronously via urllib.request with a Bearer token. All prompt construction, context injection, and response parsing happen within the FastAPI backend, keeping the frontend stateless with respect to AI logic.

2.1.2 LLM as Service Layer

Z.AI GLM functions as the reasoning service layer in two distinct agent roles within a single pipeline:

Agent 1 — SKU Extractor (Zero-shot):

- Input: Raw email text from the user
- Prompt: "Extract the specific part number/SKU from this text. Output ONLY the SKU string."
- Output: A part number string (e.g., `AE-V8-SENS`)
- Post-processing: GLM output is validated against regex pattern `[A-Z0-9]{2,}-[A-Z0-9]{2,}-[A-Z0-9]{2,}`

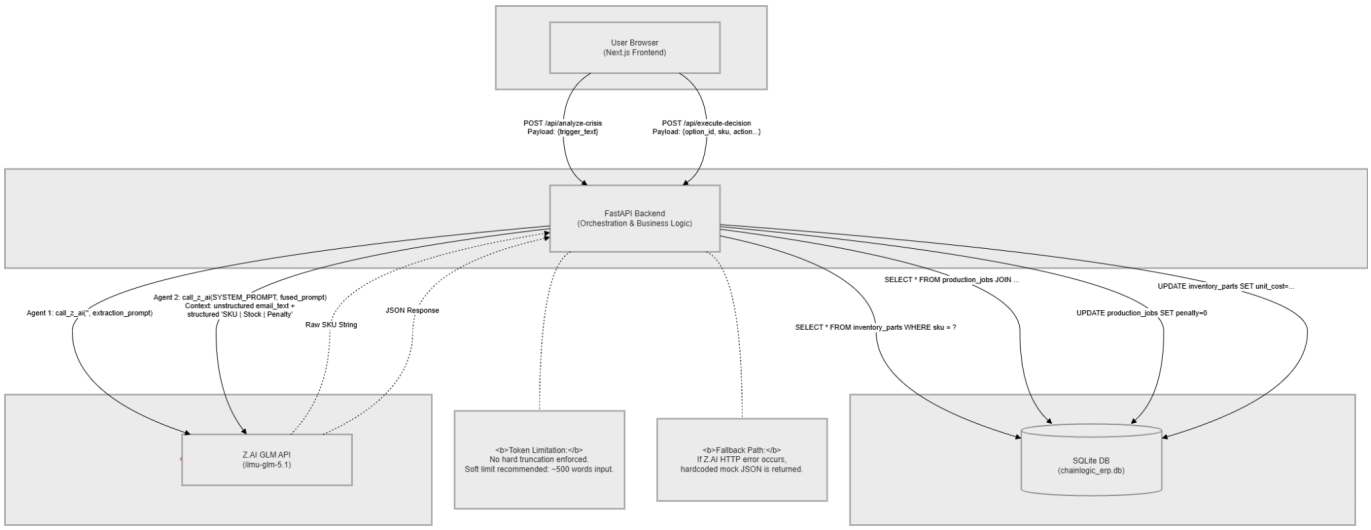
Agent 2 — Crisis Analyst (Structured System Prompt):

- Input: Original email text + live SQLite ERP context string
- System Prompt: Full `CHAINLOGIC_SYSTEM_PROMPT` defining schema, persona, and output format

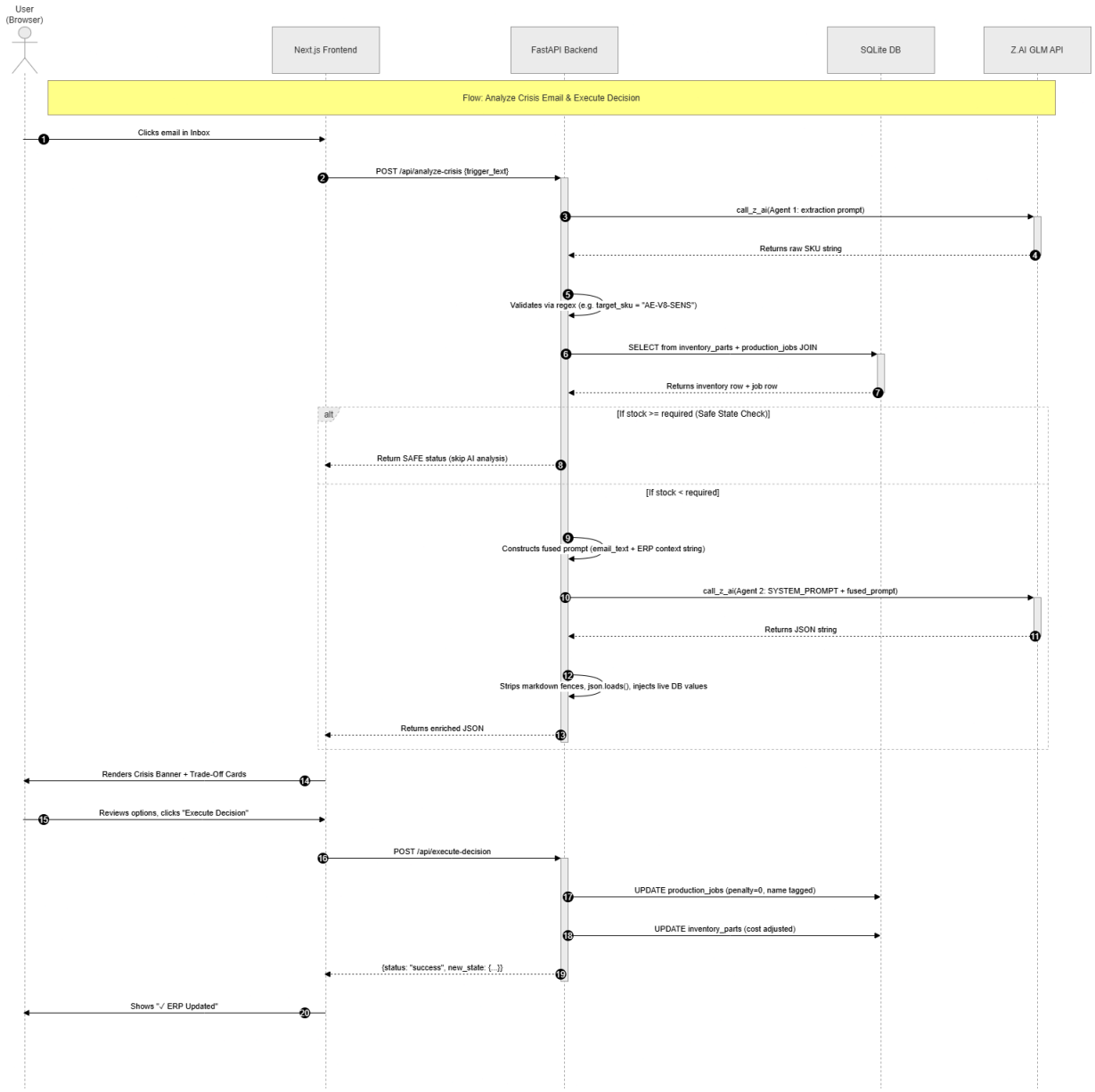
© UMHackathon 2026

- Output: Schema-conformant JSON with `crisis_analysis`, `trade_off_options`, `glm_recommendation`
- Post-processing: JSON sanitisation (markdown fence stripping) + `json.loads()` + Python data override of `baseline_impact` and `sku` fields with verified DB values

2.1.3 Dependency Diagram



2.2.2. Sequence Diagram



2.2. *Technological Stack*

Layer	*Technology*	*Justification*
Frontend	*Next.js 14 (React, TypeScript)*	*Server/client component model; fast hot-reload for hackathon iteration; TypeScript interfaces enforce API contract*

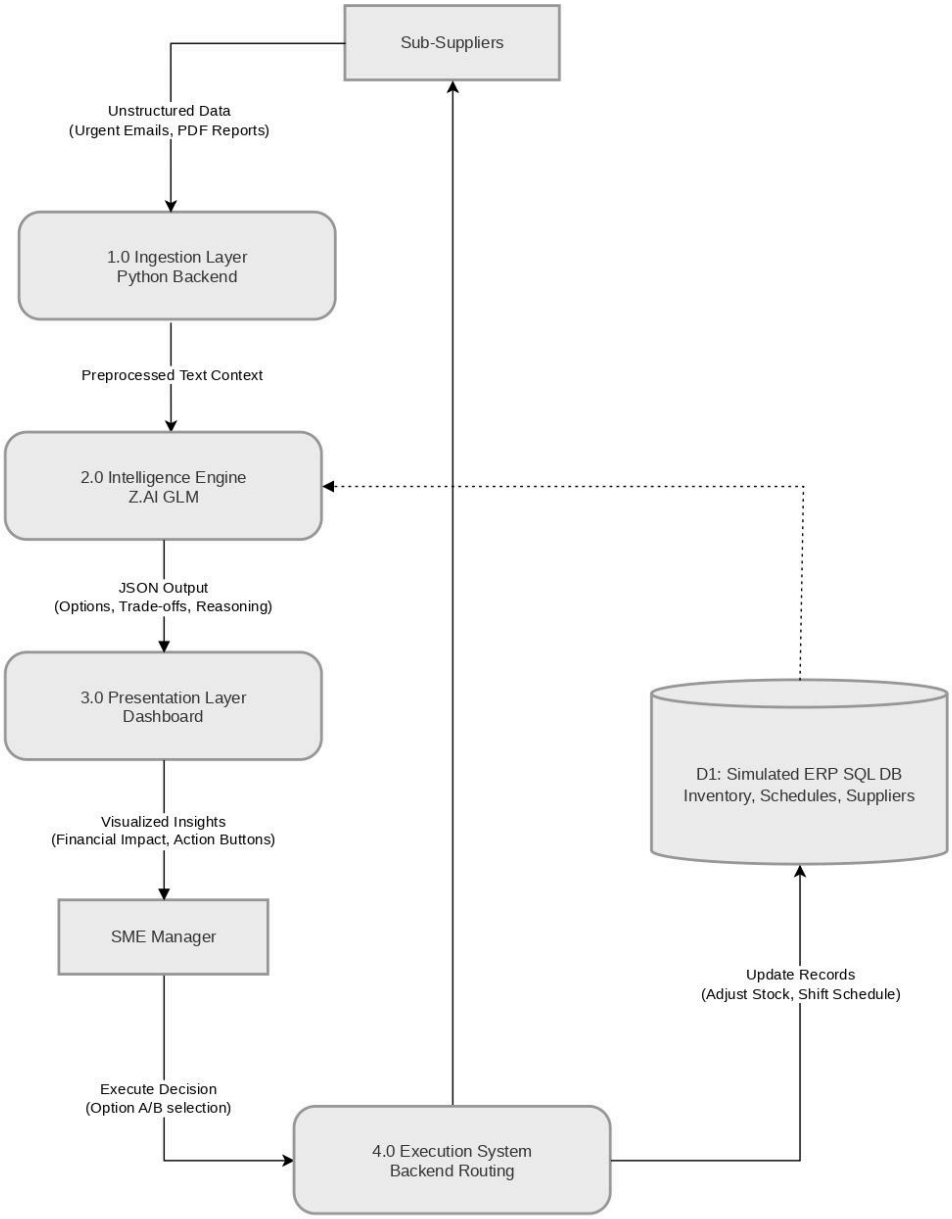
© UMHackathon 2026

Styling	*Vanilla CSS (globals.css)*	*Full design control; no Tailwind overhead; custom glassmorphism and dark-mode tokens*
UI Components	*Lucide React (icons)*	*Lightweight, tree-shakeable icon set consistent with dark UI aesthetic*
Backend	*Python FastAPI*	*Async-capable; auto-generates OpenAPI docs; minimal boilerplate for two-endpoint prototype*
AI HTTP Client	*Python urllib.request*	*Zero additional dependencies; sufficient for synchronous OpenAI-compatible POST requests*
Database	*SQLite (chainlogic_erp.db)*	*Zero-config, file-based; auto-seeded on startup; adequate for single-user hackathon demo*
Environment Config	*python-dotenv*	*Keeps API keys out of source code; loaded at runtime via os.getenv()*
AI Service	*Z.AI GLM ilmu-glm-5.1*	*OpenAI-compatible API; strong structured JSON output adherence; multilingual reasoning capability*
Dev Server	*Uvicorn (--reload) + npm run dev*	*Hot-reload on both tiers during development*

2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)



DFD Component Breakdown

1. The Actors (External Entities)

© UMHackathon 2026

Email: umhackathon@um.edu.my

Sub-Suppliers: The source of the real-world chaos (delays, price hikes) injected via unstructured text.

SME Manager: The end-user who receives the distilled intelligence and holds the final authority to execute a decision.

2. The Data Store

Simulated ERP (D1): This holds the 3NF SQL database we discussed earlier. It is the single source of truth for structured data (What do we have? Who do we buy from? When is it due?).

3. The Core Processes

Process 1.0 (Ingestion Layer): Your Python script acts as the inbox. It receives the messy PDFs and emails, strips out the junk, and formats the raw text into clean, contextual signals.

Process 2.0 (Intelligence Engine): The brain of the operation. The Z.AI GLM receives a prompt containing both the preprocessed text from Process 1.0 AND a real-time SQL query payload from D1. It computes the risk, lead times, and alternatives, outputting a strict JSON schema.

Process 3.0 (Presentation Layer): Your React/Vue/Streamlit frontend intercepts the JSON. It translates the GLM's reasoning into human-readable UI components (graphs showing financial impact, cards detailing Option A vs Option B).

Process 4.0 (Execution): When the manager clicks "Execute," this system translates that choice into backend actions—updating the SQL database to reflect new production schedules or adjusted inventory allocations.

2.3.2. **Normalized Database Schema**

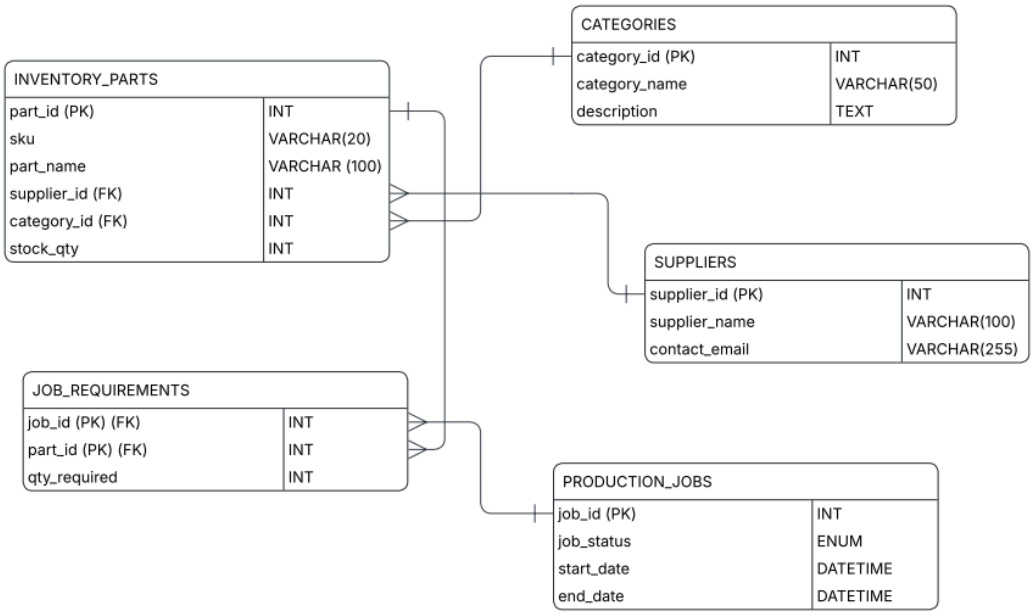
Table Definitions:

suppliers (supplier_id PK, supplier_name UNIQUE)

categories (category_id PK, category_name UNIQUE)

inventory_parts (
part_id PK,
sku UNIQUE NOT NULL,
part_name NOT NULL,
category_id FK -> categories,
supplier_id FK -> suppliers,
current_stock INT,

© UMHackathon 2026



safety_stock INT,
unit_cost REAL,
lead_time_days INT
)

production_jobs (
job_id PK,
project_name UNIQUE NOT NULL,
deadline_date TEXT,
daily_downtime_penalty REAL
)

job_requirements (
job_id FK -> production_jobs, <- Composite PK
part_id FK -> inventory_parts, <- Composite PK
required_quantity INT NOT NULL
)
3NF Justification:

© UMHackathon 2026

- *`suppliers` and `categories` are extracted into lookup tables, eliminating repeating supplier/category name strings in `inventory_parts` (resolves 2NF transitive dependency)*
- *`job_requirements` is a junction table resolving the M:N relationship between `production_jobs` and `inventory_parts` — a single job can require multiple parts; a single part can be required by multiple jobs*
- *All non-key attributes depend solely on their primary key in every table (3NF satisfied)*

Seed Data (Auto-loaded on startup):

SKU	*Part Name*	*Stock*	*Required*	*Penalty/Day*
AE-V8-SENS	*Acoustic Emission Sensor*	*12*	*40*	*$12,500*
MCU-TC397-EVO	*TriCore Microcontroller*	*150*	*1,000*	*$45,000*
BRK-PAD-99	*Ceramic Brake Pad Set*	*850*	*300*	*$2,500*
BATT-CELL-4680	*Lithium-Ion Cell 4680*	*4,000*	*8,000*	*$85,000*

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

#	*Features*	*Description*
1	*Corporate Inbox*	*Gmail-styled inbox displaying 5 realistic supplier disruption emails; user selects an email and triggers the analysis pipeline with one click*
2	*Agentic SKU Extraction*	*Z.AI GLM (Agent 1) reads the free-form email body and extracts the affected component SKU; regex fallback ensures resilience if GLM fails*
3	*AI Trade-Off Generation*	*Z.AI GLM (Agent 2) receives the fused prompt (email + live ERP context) and returns 2–3 financially quantified recovery options with formula and math breakdown*
4	*Data Pipeline Transparency Viewer*	*Expandable code-styled panel shows the exact SQLite values (stock, cost, penalty) injected into the GLM prompt — provides full auditability*
5	*Closed-Loop ERP Execution*	*User approves one option; backend autonomously updates production_jobs (penalty zeroed, action tagged) and inventory_parts (cost adjusted) in SQLite*

© UMHackathon 2026

3.2. Non-Functional Requirements (NFRs)

Quality (e.g.)	Requirements (e.g.)	Implementation (e.g)
Reliability	System must not crash or return HTTP 500 on Z.AI API failure	Three-layer fallback: regex SKU extraction -> hardcoded mock response -> structured JSON error; all exceptions caught and handled
Data Integrity	ERP execution must not write incorrect data; SKU must be validated before any DB write	SKU validated against DB before execute-decision; missing SKU returns error before any SQL runs; finally: conn.close() enforced on all DB connections
Transparency	AI-generated financial figures must be distinguishable from verified DB values	baseline_impact always overwritten by Python with real DB values; GLM computation breakdown labelled "GLM Breakdown" in UI
Token Latency	Z.AI GLM calls should return within 30 seconds under normal network conditions	Synchronous urllib.urlopen with no explicit timeout set (GLM typically responds in 15–25s); frontend loading state displayed during wait
Cost Efficiency	Avg token cost per session must be minimal	~2,360 tokens/session; Safe State short-circuit skips the expensive Agent 2 call when stock is sufficient (estimated 70–80% of non-crisis queries)
Security	API key must never appear in source code, responses, or Git history	.env excluded via .gitignore; loaded via os.getenv(); never serialised into any API response body
Maintainability	Backend logic must be self-contained and readable	All AI logic in main.py with clear function separation: call_z_ai(), extract_sku_from_trigger(), fetch_erp_context(), fetch_erp_data_dict()

3.3. Out of Scope / Future Enhancements

a. Live email integration (IMAP/SMTP): Real-time ingestion of actual supplier emails from corporate mailboxes

© UMHackathon 2026

b. *Multi-SKU disruption handling: A single email referencing multiple affected components processed in parallel*

c. *Predictive disruption forecasting: Historical trend analysis to predict future supply risks before emails arrive*

d. *Supplier outreach automation: Auto-drafting and sending RFQ emails to alternative suppliers based on AI recommendation*

e. *Multi-user authentication: Role-based access control for Supply Chain Manager, Operations Director, and Procurement Officer roles*

f. *Real ERP integration: Connectors to SAP, Oracle NetSuite, or Odoo replacing the SQLite mock*

g. *Production cloud deployment: Docker containerisation, cloud hosting (AWS/GCP), and load balancing*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1 Rollout Strategy

For post-hackathon production deployment, the recommended rollout strategy is:Phase 1 (5% of users): Deploy to a single pilot automotive SME; monitor Z.AI GLM response quality, SKU extraction accuracy, and ERP write correctness. Phase 2 (25% of users): Expand to 5 SME pilot customers; introduce A/B testing between zero-shot extraction (current) vs. few-shot extraction to determine which yields higher SKU accuracy rates. Phase 3 (100%): Full rollout once A/B test confirms extraction accuracy >= 95% and no P0/P1 bugs are open

4.1.2 Emergency Rollback & Golden Release

Trigger: Z.AI GLM API error rate exceeds 20% over a 5-minute window, or ERP write failure rate exceeds 1%. Action: Automated rollback to the last stable "Golden Release" tag in the Git repository; the hardcoded mock fallback activates automatically for all active sessions, ensuring users receive a usable (if non-live) response. Recovery: Root cause identified; fix deployed to staging; integration tests re-run; manual promotion to Golden Release after 24-hour stability window

4.1.3 Priority Matrix:

Priority	Condition	Action

© UMHackathon 2026

P0 — Critical	*Z.AI GLM returns errors for >50% of requests for >10 minutes*	*Immediate emergency rollback; all sessions fall through to hardcoded mock; team notified*
P1 — High	*ERP execute-decision writes incorrect SKU or penalty value*	*Halt execution endpoint; revert DB to last known good state; hotfix deployed*
P1 — High	*API key exposed in any response or Git commit*	*Immediate key rotation; commit history scrubbed; deploy with new key*
P2 — Medium	*SKU extraction accuracy drops below 90% (measured over 50 requests)*	*Switch extraction to regex-only mode; queue GLM prompt tuning investigation*
P3 — Low	*Frontend error messages displayed as raw JSON (cosmetic)*	*Log to backlog; fix in next sprint; no user data at risk*

4.2. Monitoring:

Signal	*Metric*	*Threshold*	*Alert Action*
Z.AI API health	*HTTP error rate on GLM calls*	*> 10% over 5 min*	*Log warning; trigger fallback mock automatically*
SKU extraction accuracy	*% of GLM responses that pass regex validator*	*< 90%*	*Switch to regex-only mode; log for prompt review*
ERP write integrity	*DB row diff before/after execute-decision*	*Mismatch on penalty or sku field*	*Rollback DB write; return error to user*
API response time	*End-to-end latency of /api/analyze-crisis*	*> 40 seconds*	*Log timeout; return fallback mock; alert team*
Security scan	*Git commit diff for .env or key string patterns*	*Any match*	*Block push; alert immediately*

4.3. Assumptions:

1. *The Z.AI GLM API (`api.ilmu.ai`) is available and responsive during the hackathon demo window*
2. *Supplier disruption emails contain at least one component SKU matching the pattern `[A-Z0-9]{2,}-[A-Z0-9]{2,}-[A-Z0-9]{2,}`*

© UMHackathon 2026

3. *The SQLite database is running on the same machine as the FastAPI backend with no network latency for DB queries*
4. *The demo is conducted by a single user — no concurrent session management is required*
5. *The hackathon judges will access the system on a machine with Python 3.9+ and Node.js 18+ already installed*
6. *The `.env` file with the valid `Z_AI_API_KEY` is present on the demo machine (not committed to Git)*
7. *Network connectivity to `api.ilmu.ai` is stable during the demo*

4.4. External Dependencies:

Tool / Service	*Purpose*	*Risk Level*	*Mitigation*
Z.AI GLM API (ilmu-glm-5.1)	*Core LLM reasoning engine — SKU extraction (Agent 1) and financial trade-off generation (Agent 2)*	*High — if the API is unavailable, Agents 1 and 2 both fail*	*Three-layer fallback: regex extraction -> hardcoded mock response -> structured error. System never returns HTTP 500 to the user*
SQLite	*Persistent ERP data store — inventory, production jobs, suppliers, categories*	*Low — file-based, no network dependency*	*Auto-seeded on startup; finally: conn.close() prevents lock accumulation*
python-dotenv	*Loads .env variables at runtime*	*Low — pure Python package, no external API*	*os.getenv() defaults set for all variables so server starts even without .env*
Lucide React	*UI icon set*	*Negligible — npm package, no external API*	*Bundled at build time; no runtime network call*
Next.js / Uvicorn	*Development servers*	*Low — standard open-source tools*	*Standard package managers; pinned versions in package.json and requirements.txt*

5. Project Management & Team Contributions

5.1. Project Timeline

Day(s)	Phase	Deliverables
Day 1–2	Planning & Architecture	System design decisions; DB schema design; API contract definition; Z.AI GLM prompt strategy
Day 3	Backend Foundation	FastAPI server setup; SQLite schema + seed data; call_z_ai() function; /api/analyze-crisis skeleton
Day 4	AI Integration	Two-step agentic pipeline (extraction + analysis); JSON sanitisation; fallback layers; Safe State logic
Day 5	Frontend Development	Next.js inbox view; trade-off card components; Data Pipeline Viewer; execute-decision integration
Day 6	Integration & ERP Execution	/api/execute-decision endpoint; DB write verification; end-to-end flow testing
Day 7	Testing & Bug Fixes	QATD test case execution; defect fixes; adversarial prompt testing; performance checks
Day 8	Demo Preparation & Documentation	PRD, QATD, SAD finalisation; demo script rehearsal; README updated

5.2. Team Members Role:

Member	Role	Responsibilities
JULIUS LIM JUN HERNG LAI ZI LI	Architecture & Integration	System design; FastAPI backend; Z.AI GLM prompt engineering; agentic pipeline orchestration
LAI ZI LI CHIN KIN HIUNG	React / Next.js UI	Inbox view; trade-off card components; loading states; CSS design system

© UMHackathon 2026

WONG HAO LEONG *IVAN CHAN JU HONG*	*Database & API*	*SQLite schema design; seed data; ERP execution endpoint; fallback logic*
JULIUS LIM JUN HERNG	*Testing & Docs*	*QATD execution; PRD and SAD writing; defect logging; demo script preparation*

5.3. Recommendations

Area	*Recommendation*	*Rationale*
Performance	*Add a response cache (e.g., Redis) keyed on hash(email_text + sku) with a 1-hour TTL*	*Identical disruption emails submitted multiple times (e.g., team members reviewing the same alert) would return instantly without consuming Z.AI tokens*
Scalability	*Migrate from SQLite to PostgreSQL for multi-user deployment*	*SQLite write locks become a bottleneck beyond a single concurrent user; PostgreSQL supports row-level locking and connection pooling*
Reliability	*Add an explicit timeout=30 parameter to urllib.urlopen()*	*Currently, a hung GLM response would block the FastAPI thread indefinitely; a 30s timeout triggers the fallback cleanly*
Security	*Rotate Z_AI_API_KEY before any public deployment or Git push review*	*The current key in .env should be treated as demo-only; production keys should be stored in a secrets manager (e.g., AWS Secrets Manager)*
UX	*Add a frontend character counter on the email textarea with a soft 500-word warning*	*Users have no current feedback on input size; oversized inputs increase GLM latency significantly*

© UMHackathon 2026

Observability	*Integrate structured logging (e.g., Python logging module with JSON formatter)*	*Current print() statements are not queryable; structured logs enable filtering by SKU, session, or error type for post-demo analysis*

© UMHackathon 2026